

RxJS Deep Dive

These are the **10 most used** in real Angular + .NET backend applications.

- ✓ 1. Observable
- ✓ 2. Observer
- ✓ 3. Subscription
- ✓ 4. Operators vs Pipe
- ✓ 5. Subject
- ✓ 6. BehaviorSubject
- ✓ 7. ReplaySubject
- ✓ 8. AsyncSubject
- ✓ 9. Hot vs Cold Observables
- ✓ 10. Unsubscribing patterns

★ 1. Observable

Interview Definition

An **Observable** is a producer of values over time. You get the values using `subscribe()`.

When used in real projects?

- ✓ HTTP calls
- ✓ WebSocket events
- ✓ Search boxes
- ✓ Route params
- ✓ Form value changes

Example

```
this.http.get('/api/products'); // returns Observable<Product[]>
```

★ 2. Observer

The object with `next`, `error`, `complete`.

Example

```
obs.subscribe({  
  next: v => console.log(v),  
  error: e => console.error(e),  
  complete: () => console.log('done')  
});
```

RxJS Deep Dive

Interview question

Q: What are the three methods of an observer?

A: next, error, complete.

★ 3. Subscription

The result of `.subscribe()`.

Example

```
const sub = obs.subscribe();
```

```
sub.unsubscribe();
```

Real project scenario

You must **unsubscribe** when:

- Component destroys
 - Long-running streams (interval, WebSocket)
-

★ 4. Operators + pipe()

Operators $\neq$ functions.

Operators **modify** an Observable.

Example (pipe)

```
this.http.get("/api/users")
```

```
  .pipe(  
    map(res => res.data),  
    filter(x => x.active)  
  )
```

```
  .subscribe();
```

Types of Operators

- Transformation $\rightarrow$ map, switchMap, concatMap, mergeMap
 - Filtering $\rightarrow$ filter, take, takeUntil, distinctUntilChanged
 - Combining $\rightarrow$ forkJoin, combineLatest, withLatestFrom
-

★ 5. Subject

Subject = Observable + Observer at same time.

RxJS Deep Dive

When used?

- ✓ Event emitters
- ✓ Sharing data between components
- ✓ Manual push of values

Example

```
const click$ = new Subject();  
click$.next("button clicked");  
click$.subscribe(v => console.log(v));
```

Common mistake interviewer asks:

“Does Subject store last value?”

✗ No.

★ 6. BehaviorSubject (Most asked in interviews!)

Interview Definition

BehaviorSubject stores **latest value** and immediately sends it to new subscribers.

Real project use

- ✓ Authentication state
- ✓ Cart
- ✓ Filters
- ✓ Theme state
- ✓ Notification counter

Example

```
user$ = new BehaviorSubject<User | null>(null);
```

```
login(user) {  
  this.user$.next(user);  
}
```

```
constructor() {  
  this.user$.pipe(  
    filter(user => !!user)  
  ).subscribe();  
}
```

RxJS Deep Dive

★ 7. ReplaySubject

Stores **buffer** of past values and replays them to new subscribers.

Real project use

When new subscribers must get **history**, not just last value.

Example:

- ✓ Chat messages
- ✓ Audit logs
- ✓ Notification stream

```
msgs$ = new ReplaySubject<string>(3); // store last 3 messages
msgs$.next("Hi");
msgs$.next("Hello");
msgs$.next("How are you?");
msgs$.subscribe(console.log);
// subscriber receives all 3
```

★ 8. AsyncSubject

Emits **only the last value** when .complete() is called.

Real project use

- ✓ API calls wrapped in custom Observables
- ✓ Long computations
- ✓ File uploads where only final status matters

Example

```
const data$ = new AsyncSubject();
data$.next(1);
data$.next(2);
data$.next(3);

data$.complete(); // emits only "3"
```

★ 9. Hot vs Cold Observables (INTERVIEW HOT TOPIC 🔥)

Cold Observable

Each subscriber gets **fresh execution**.

RxJS Deep Dive

Example:

`this.http.get(...)` // runs again for every subscriber

Hot Observable

Shared source; all subscribers receive **same values**.

Examples:

- Subject
- BehaviorSubject
- ReplaySubject
- DOM events

Interview Line:

“HTTP in Angular is cold, Subjects are hot.”

★ 10. Unsubscribing Patterns

Best practices

- Components → use `takeUntil(this.destroy$)`
- Services → usually no unsubscribe needed
- Use async pipe in template

Example

```
destroy$ = new Subject<void>();
```

```
ngOnInit() {  
  this.userService.data$  
    .pipe(takeUntil(this.destroy$))  
    .subscribe();  
}  
  
ngOnDestroy() {  
  this.destroy$.next();  
  this.destroy$.complete();  
}
```

RxJS Deep Dive

HIGH PRIORITY OPERATORS

- map
- filter
- tap
- debounceTime
- distinctUntilChanged
- switchMap ★★★★★
- mergeMap
- concatMap
- exhaustMap
- take, takeUntil
- shareReplay
- forkJoin
- combineLatest
- withLatestFrom

switchMap() – Most Important Operator in Angular

Simple meaning:

Cancels the previous request and switches to a new one.

Why we use it:

Used in **search, filter, typeahead, dropdown dependencies**, etc.

Real Example (Billing System):

When typing Customer Name in search box:

```
fromEvent(inputBox, 'keyup').pipe(  
  debounceTime(300),  
  switchMap(value => this.api.getCustomers(value))  
)  
.subscribe(result => console.log(result));
```

- ✓ Only latest typed value will call API
- ✓ Prevents multiple unnecessary API calls
- ✓ Very common in interviews

RxJS Deep Dive

mergeMap() – Parallel Requests

Simple meaning:

Runs all Observables **in parallel**.
Does NOT cancel previous ones.

Use case:

Upload multiple files
Send multiple invoices
Fetch extra details for a list

Example:

```
from(files).pipe(  
  mergeMap(file => this.api.upload(file))  
)  
  
.subscribe(response => console.log(response));
```

- ✓ Perfect for batch processing
- ✓ Not good for auto-complete (use switchMap instead)

concatMap() – Executes One by One

Simple meaning:

Runs Observables **sequentially** (FIFO).

Use case:

When order matters:

- Create Invoice → Then send Email
- Add items → Then update stock
- Log → Then update DB

Example:

```
from(tasks).pipe(  
  concatMap(task => this.api.process(task))  
)  
  
.subscribe();
```

- ✓ Guarantees order
- ✓ Slower than mergeMap

RxJS Deep Dive

exhaustMap() – Ignore New Requests Until Previous Completed

Simple meaning:

“Busy right now, come later.”

Use case:

- Prevent multiple button clicks
- Avoid duplicate payments
- Avoid double invoice submission

Example:

```
fromEvent(btn, 'click').pipe(  
  exhaustMap(() => this.api.submitInvoice())  
)  
  
.subscribe();
```

- ✓ Prevents double form submission
 - ✓ Very useful in payment/checkout modules
-

forkJoin() – Wait Until All Requests Finish

Simple meaning:

Runs requests in parallel → waits for all → returns once.

Use case (Billing App):

Load Invoice Page:

- Customer details
- Invoice items
- Taxes
- Discounts

Example:

```
forkJoin({  
  customer: this.api.getCustomer(id),  
  items: this.api.getInvoiceItems(id),  
  taxes: this.api.getTaxes()  
})  
  
.subscribe(result => console.log(result));
```

RxJS Deep Dive

- ✓ Best for page initialization
 - ✓ Interviewers ask forkJoin almost always
-

combineLatest() – Get Latest Values from Multiple Sources

Simple meaning:

Whenever ANY observable changes, emit a combined result.

Use case:

- Filters (Date + Status + Amount)
- Dashboard metrics
- Live search + category filter

Example:

```
combineLatest([
  this.dateFilter$,
  this.statusFilter$,
  this.searchText$
])
.subscribe(([date, status, text]) => {
  this.applyFilters(date, status, text);
});
```

- ✓ Perfect for dynamic filtering
 - ✓ Very common in real dashboards
-

withLatestFrom() – Combine but Only When Primary Observable Emits

Use case:

- Press search button → get latest filters
- Submit → take latest form values

Example:

```
this.searchBtnClick$.pipe(
  withLatestFrom(this.filter$)
)
.subscribe(([click, filter]) => {
```

RxJS Deep Dive

```
this.loadData(filter);
```

```
});
```

✓ Best when you don't want continuous updates

✓ Interview favorite

takeUntil() – Clean Unsubscribe Automatically

Use case:

- Destroy component
- Cancel a long-running stream
- Auto-unsubscribe pattern

Pattern Example:

```
private destroy$ = new Subject<void>();
```

```
this.api.getInvoices().pipe(  
  takeUntil(this.destroy$)  
)
```

```
ngOnDestroy() {  
  this.destroy$.next();  
  this.destroy$.complete();  
}
```

✓ Required in interview

✓ Required for memory leak prevention

shareReplay() – Cache API Result

Use case:

- Avoid calling API multiple times
- Shared data: user, settings, config
- Dashboard load optimization

Example:

```
settings$ = this.api.getSettings().pipe(  
  shareReplay(1)
```

RxJS Deep Dive

```
shareReplay(1)

);
```

- ✓ Saves backend load
 - ✓ Boosts performance
 - ✓ Common real-project pattern
-

catchError() – Handle API Errors Gracefully

Effect:

```
this.api.getInvoices().pipe(
  catchError(err => {
    this.toast.error("Unable to load invoices");
    return of([]);
  })
)
```

- ✓ Required for user-friendly applications
 - ✓ Avoids frontend crashes
 - ✓ High interview importance
-

retry() & retryWhen()

retry(): Automatically try API again

```
this.api.getData().pipe(
  retry(3)
)
```

retryWhen(): Custom retry logic

(Optional delays/backoff)

debounceTime() – Control API Spamming

Use case:

- Search
 - Filter
 - Auto-suggest
 - Typeahead
-

RxJS Deep Dive

```
this.search$.pipe(  
  debounceTime(400),  
  switchMap(text => this.api.search(text))  
)
```

- ✓ Mandatory operator in search scenarios
 - ✓ Always asked in interviews
-

distinctUntilChanged() – Prevent Duplicate API Calls

Avoid calling API when value did not change.

```
this.search$.pipe(  
  distinctUntilChanged(),  
  switchMap(value => this.api.search(value))  
)
```

- ✓ Saves backend
 - ✓ Used with form inputs
-

tap() – Logging and Debugging

tap does NOT change value

Used to:

- Show loader
- Hide loader
- Log result

```
this.api.getInvoices().pipe(  
  tap(() => this.loader = true)  
)  
  
.subscribe();
```

- ✓ Helps debugging streams
 - ✓ Very useful in real projects
-

RxJS Deep Dive

map() vs switchMap()

Simple Interview Answer:

map	switchMap
Used to transform data	Used to transform AND call new Observable
Does NOT cancel streams	Cancels previous stream
Synchronous	Async-friendly

Hot vs Cold Observables – Simple Explanation

Cold Observable

Creates a **new execution** for each subscriber.

Examples:

- http.get()
- from([1,2,3])
- interval() (until subscribed)

Every subscriber gets **its own data**.

Hot Observable

Data **already running**, subscribers join in the middle.

Examples:

- Subject
- WebSocket feed
- Mouse events

Simple analogy (interview-friendly)

👉 Cold = Watching movie from the beginning

👉 Hot = Joining a live cricket match (you don't see what already happened)

Subject vs BehaviorSubject vs ReplaySubject vs AsyncSubject

Subject

- No initial value
- New subscribers get **nothing until next emit**

```
const sub = new Subject();
```

RxJS Deep Dive

`sub.next(1);` // pushed to current subscribers only

Use case:

- Button clicks
 - Events
-

BehaviorSubject

- Requires initial value
- New subscribers **immediately** receive latest value

```
const bs = new BehaviorSubject(0);
```

```
bs.subscribe(v => console.log(v)); // 0
```

Use case:

- Login state
 - Cart state
 - Theme settings
-

ReplaySubject

- Stores **past values** (buffer size)

```
const rs = new ReplaySubject(2); // stores last 2 values
```

Use case:

- Chat messages
 - Activity logs
 - Storing last n events
-

AsyncSubject

- Emits **only the last value** when completed

```
const as = new AsyncSubject();
```

```
as.next(1);
```

```
as.next(2);
```

```
as.complete(); // emits 2 only
```

Use case:

- API caches
-

RxJS Deep Dive

- One-time operations
- Final result computations

Higher-Order Observables

Observables that emit Observables.

Example:

HTTP request → return another HTTP request

```
this.userId$.pipe(  
  switchMap(id => this.api.getUser(id)),  
  switchMap(user => this.api.getOrders(user.id))  
)
```

- ✓ Interviewers love higher-order pipeline questions
- ✓ switchMap is MOST used for higher-order cases

Creating Custom RxJS Operators

Custom operator = reuse logic across pipes.

Example – Loader operator (real project use)

```
function withLoader(loader: Signal<boolean>) {  
  return (source) => source.pipe(  
    tap(() => loader.set(true)),  
    finalize(() => loader.set(false))  
  );  
}
```

Usage:

```
this.api.getInvoices().pipe(  
  withLoader(this.loading)  
)  
  
.subscribe();
```

- ✓ Reduces repeated loader code
- ✓ Great for clean coding questions
- ✓ Impressive in interviews

RxJS Deep Dive

Unsubscribing Patterns (Best Practices)

Option 1 – takeUntil (most common)

```
private destroy$ = new Subject();
```

```
obs.pipe(takeUntil(this.destroy$)).subscribe();
```

Option 2 – Async pipe (recommended)

```
<div *ngFor="let item of items$ | async">{{ item }}</div>
```

- ✓ Auto unsubscribe
- ✓ Best for components

Multicasting with share() and shareReplay()

share()

Prevents re-running execution for multiple subscribers.

shareReplay(1)

Caches last emitted value.

Real Example:

```
settings$ = this.api.getSettings()
```

```
.pipe(shareReplay(1));
```

- ✓ Avoids calling API multiple times
- ✓ Perfect for shared services

debounceTime + distinctUntilChanged (Search Optimization)

Real search box:

```
searchControl.valueChanges.pipe(  
  debounceTime(300),  
  distinctUntilChanged(),  
  switchMap(text => this.api.searchCustomer(text))  
)  
.subscribe();
```

- ✓ This combo is asked in 80% Angular interviews.

backoff Retry Strategy (Advanced retryWhen)

RxJS Deep Dive

```
retryWhen(errors =>
  errors.pipe(
    scan((count) => count + 1, 0),
    delayWhen(count => timer(count * 1000)), // 1s, 2s, 3s
    takeWhile(count => count < 4)
  )
)
```

- ✓ Real production code
- ✓ Advanced interview topic

Parallel API Calls with mergeMap + forkJoin Pattern

Example: Invoices + Items + Taxes + Customer details

```
invoiceIds$.pipe(
  mergeMap(id => forkJoin({
    invoice: this.api.getInvoice(id),
    items: this.api.getItems(id),
    taxes: this.api.getTaxes()
  }))
)
.subscribe(result => console.log(result));
```

- ✓ Common in Billing / ERP systems

Caching with Timestamp (Smart Cache Pattern)

```
cache$: Observable<any>;
lastFetch = 0;

getData() {
  const now = Date.now();
  if (!this.cache$ || now - lastFetch > 60000) {
    lastFetch = now;
    this.cache$ = this.api.data().pipe(shareReplay(1));
  }
}
```

RxJS Deep Dive

```
}  
  
return this.cache$;  
  
}
```

- ✓ Prevents 100s of API calls
 - ✓ Real enterprise-level optimization
-

race() – Pick the Fastest Observable

Use case:

- Load data from multiple servers and take fastest
- Fallback API calls

```
race(  
  this.api.getFromServerA(),  
  this.api.getFromServerB()  
)  
  
.subscribe(fastest => console.log(fastest));
```

✓ Smart failover pattern

zip() – Pair Observables Together

Emits when ALL observables emit one value each.

```
zip(this.amount$, this.tax$)  
  
.subscribe(([amount, tax]) => console.log(amount * tax));
```

✓ Great for paired data calculations

bufferTime / bufferCount

Collect values into groups.

Example:

Batch logging every 5 seconds

```
events$.pipe(  
  bufferTime(5000)  
)  
  
.subscribe(batch => console.log(batch));
```

RxJS Deep Dive

- ✓ Useful for batching analytics
- ✓ Real production use

throttleTime() – Limit Clicks

Prevents spam clicking.

```
fromEvent(btn, 'click').pipe(  
  throttleTime(2000)  
)
```

- ✓ Good for submit buttons
- ✓ Asked in async-control questions

pairwise() – Compare Previous vs Current Value

```
control.valueChanges.pipe(pairwise())  
.subscribe(([prev, curr]) => console.log(prev, curr));
```

Use case:

- Find user input patterns
- Compare last value with new one
- Detect form changes

Real-time WebSocket with RxJS (Angular + .NET 8 patterns)

Key ideas (interview bullets)

- Use RxJS WebSocketSubject for client streams or build your own Subject wrapper for reconnection/backoff.
- Keep socket as a singleton service.
- Use share()/shareReplay() so multiple components reuse the single connection.
- Handle reconnection, heartbeats/ping, and graceful close.
- Server: use WebSocket middleware in .NET to accept connections and broadcast events (or use SignalR for full-featured stack — but below shows raw WebSocket pattern).

Angular — WebSocket service (with reconnection/backoff)

```
// ws.service.ts
```

```
import { Injectable } from '@angular/core';
```

```
import { WebSocket, WebSocketSubject } from 'rxjs/webSocket';
```

RxJS Deep Dive

```
import { Observable, timer, Subject, BehaviorSubject, merge } from 'rxjs';  
import { retryWhen, delayWhen, tap, filter, switchMap } from 'rxjs/operators';
```

```
@Injectable({ providedIn: 'root' })  
export class WsService {  
  private url = 'wss://your-api.com/ws';  
  private socket$: WebSocketSubject<any>;  
  private connectionStatus = new BehaviorSubject<boolean>(false);  
  public messages$ = new Subject<any>(); // broadcast messages to app  
  
  private reconnectAttempts = 0;  
  
  constructor() {  
    this.connect();  
  }  
  
  private createSocket(): WebSocketSubject<any> {  
    return websocket({  
      url: this.url,  
      openObserver: { next: () => {  
        this.reconnectAttempts = 0;  
        this.connectionStatus.next(true);  
      }  
    },  
      closeObserver: { next: () => {  
        this.connectionStatus.next(false);  
      }  
    }  
  });  
  }  
}
```

RxJS Deep Dive

```
private connect() {  
  this.socket$ = this.createSocket();  
  
  this.socket$  
    .pipe(  
      retryWhen(errors =>  
        errors.pipe(  
          tap(err => {  
            this.reconnectAttempts++;  
            console.warn('WebSocket error, attempt', this.reconnectAttempts);  
          }  
        ),  
        // exponential backoff with max delay  
        delayWhen(() => timer(Math.min(1000 * (2 ** this.reconnectAttempts), 30000)))  
      )  
    )  
    .subscribe({  
      next: msg => this.messages$.next(msg),  
      error: err => console.error('socket error', err),  
      complete: () => {  
        console.warn('socket closed');  
        // connectionStatus false handled in closeObserver  
      }  
    });  
}  
  
// send message to server  
send(msg: any) {  
  if (this.socket$ && !this.socket$.closed) {  
    this.socket$.next(msg);  
  } else {
```

RxJS Deep Dive

```
        console.warn('socket not connected');
    }
}

// expose connection status
isConnected(): Observable<boolean> {
    return this.connectionStatus.asObservable();
}

close() {
    this.socket$.complete(); // closes socket
}
}
```

Component usage (subscribe safely):

```
constructor(private ws: WsService) {
    ws.messages$.subscribe(msg => this.handle(msg));
}
```

.NET 8 minimal WebSocket example (raw, interview-level)

Production tip: For scale, prefer **SignalR** or broker + worker pattern (Redis/RabbitMQ) — shown later in architecture.

```
// Program.cs (minimal API)
var builder = WebApplication.CreateBuilder(args);
var app = builder.Build();

app.MapGet("/ws", async (HttpContext context) =>
{
    if (context.WebSockets.IsWebSocketRequest)
    {
        using var socket = await context.WebSockets.AcceptWebSocketAsync();
        var buffer = new byte[1024 * 4];
```

RxJS Deep Dive

```
var result = await socket.ReceiveAsync(new ArraySegment<byte>(buffer),
CancellationToken.None);

while (!result.CloseStatus.HasValue)
{
    // echo back (example)

    await socket.SendAsync(new ArraySegment<byte>(buffer, 0, result.Count),
result.MessageType, result.EndOfMessage, CancellationToken.None);

    result = await socket.ReceiveAsync(new ArraySegment<byte>(buffer),
CancellationToken.None);
}

await socket.CloseAsync(result.CloseStatus.Value, result.CloseStatusDescription,
CancellationToken.None);
}
else
{
    context.Response.StatusCode = 400;
}
});

app.UseWebSockets();

app.Run();
```

Note (interview): This accepts a single socket per request. For broadcasting to many clients, maintain a concurrent collection of sockets in a singleton service and send messages to all connected sockets.

Extra patterns to mention in interviews

- Heartbeat/ping: client sends periodic ping, server responds; disconnect on missing pongs.
- Authorization: pass JWT in query param or initial HTTP header handshake and validate server-side.
- Use `share()` or `shareReplay(1)` if exposing the socket observable so multiple subscribers don't create separate connections.
- For complex apps use **SignalR** (handles reconnection/auth, scaling via backplane like Redis).

2) State management using RxJS (without NgRx) — Service-based store pattern

RxJS Deep Dive

Goal

A small, testable, scalable store using RxJS primitives (no NgRx): keep state in a BehaviorSubject, expose observables (selectors), and provide actions (methods) + optional effects for side effects.

Minimal store pattern (example: InvoiceStore)

```
// invoice-store.service.ts

import { Injectable } from '@angular/core';
import { BehaviorSubject, Observable } from 'rxjs';
import { map, distinctUntilChanged } from 'rxjs/operators';

export interface InvoiceState {
  list: Invoice[];
  selectedId: number | null;
  loading: boolean;
  error: string | null;
}

const initialState: InvoiceState = {
  list: [],
  selectedId: null,
  loading: false,
  error: null
};

@Injectable({ providedIn: 'root' })
export class InvoiceStore {
  private _state = new BehaviorSubject<InvoiceState>(initialState);
  readonly state$ = this._state.asObservable();

  // Selectors
  readonly invoices$ = this.state$.pipe(map(s => s.list), distinctUntilChanged());
```

RxJS Deep Dive

```
readonly selectedInvoice$ = this.state$.pipe(
  map(s => s.list.find(i => i.id === s.selectedId) || null),
  distinctUntilChanged()
);

readonly loading$ = this.state$.pipe(map(s => s.loading), distinctUntilChanged());

constructor(private api: InvoiceApiService) {}

// Actions
private setState(patch: Partial<InvoiceState>) {
  this._state.next({ ...this._state.value, ...patch });
}

loadAll() {
  this.setState({ loading: true, error: null });
  this.api.getInvoices().subscribe({
    next: list => this.setState({ list, loading: false }),
    error: err => this.setState({ loading: false, error: err.message || 'Load error' })
  });
}

select(id: number) {
  this.setState({ selectedId: id });
}

add(invoice: Invoice) {
  // optimistic update example:
  const current = [...this._state.value.list];
  this.setState({ list: [...current, invoice] });

  this.api.create(invoice).subscribe({
```

RxJS Deep Dive

```
next: saved => {  
  // replace temp with saved  
  const list = this._state.value.list.map(i => i === invoice ? saved : i);  
  this.setState({ list });  
},  
error: () => {  
  // rollback  
  this.setState({ list: current });  
}  
});  
}  
}
```

Good practices (interview bullets)

- Expose readonly observables — never give direct access to BehaviorSubject.
- Keep state immutable — always produce new arrays/objects.
- Use selectors for derived data (e.g., totals) via map() or combineLatest.
- Effects: keep side effects (HTTP) inside store or separate effects service.
- For caching: use shareReplay(1) on API observables and a TTL strategy.

3) Real interview coding tasks (with solutions & explanation)

I'll give 4 common tasks that interviewers love. Each includes a short explanation + code.

Task A — Debounced search with cancelation (typeahead)

Problem: Implement input search that waits 300ms and cancels previous requests.

Solution (component):

```
searchControl = new FormControl();
```

```
ngOnInit() {  
  this.searchControl.valueChanges.pipe(  
    debounceTime(300),  
    distinctUntilChanged(),
```

RxJS Deep Dive

```
filter(q => q && q.length >= 2),  
switchMap(q => this.api.searchCustomers(q))  
).subscribe(results => this.results = results);  
}
```

Why: switchMap cancels previous pending HTTP calls — prevents race conditions.

Task B — Exponential retry for flaky API

Problem: Retry up to 4 times with exponential backoff (1s, 2s, 4s, 8s).

Solution:

```
import { retryWhen, scan, delay } from 'rxjs/operators';  
this.api.getData().pipe(  
  retryWhen(errors => errors.pipe(  
    scan((acc, value) => acc + 1, 0),  
    delay((count) => Math.pow(2, count) * 1000),  
    takeWhile(count => count < 4)  
  ))  
)  
.subscribe();
```

Why: resilient network calls; interviewer looks for retryWhen + backoff logic.

Task C — Real-time notifications consumer with reconnection

Problem: Subscribe to WebSocket notifications and keep connection alive.

Solution: (see WsService earlier). Key points:

- Wrap WebSocketSubject in a service.
- Use retryWhen for reconnection.
- Expose messages\$ Subject for app components to subscribe.

Edge cases to mention: replay missed messages on reconnect (use server side seq ids), authenticate socket, and use backplane (Redis) for multiple server instances.

Task D — Implement a small store without NgRx (actions + selectors)

Problem: Build a store that supports add, remove, select and getTotal.

RxJS Deep Dive

Solution: See InvoiceStore above. For interviews you can briefly show add(), remove() using immutable updates and show total\$ = invoices\$.pipe(map(list => list.reduce(...))).

Why interviewers like it: Shows understanding of state immutability, selectors, and separation of concerns.

4) End-to-end architecture diagrams & flows — Billing System (Angular + .NET 8)

Below is a compact architecture and a few typical flows. Use these to explain how all pieces fit in interviews.

Components (short list)

- Angular (client): Components, Services (RxJS/Signals), WebSocket client, Auth guard.
 - API (.NET 8 REST): Controllers, Services, Repositories, Background workers.
 - Real-time: WebSocket endpoint or SignalR Hub.
 - DB: SQL Server / PostgreSQL
 - Message Broker: Redis / RabbitMQ for scaling notifications / background jobs
 - Cache: Redis for shared cache / session store
 - File storage: S3 or Azure Blob
 - CI/CD: GitHub Actions/Azure DevOps
 - Monitoring: Prometheus/ElasticStack
-

ASCII diagram (high-level)

[Angular Client]

- HTTP (REST) ---> [API Gateway / .NET 8 REST]

- WS (wss://) ---> [WebSocket/SignalR Hub] ---> [Message Broker/Backplane]

\

-> [Notification Worker] -> DB / Push

[API] <----> [DB] (SQL)

[API] <----> [Message Broker] (publish events)

[Background Workers] <----> [Message Broker] (consumes)

[Cache: Redis] used by API and Workers for shared state

[Auth: JWT/OAuth] - tokens issued by Auth service

RxJS Deep Dive

Typical flow: Create Invoice → Real-time update for Dashboard

1. UI calls POST /api/invoices (HTTP).
2. API persists invoice to DB and publishes invoice.created event to message broker.
3. Background worker (or API itself) pushes message to WebSocket hub (or SignalR) — either directly or via broker backplane.
4. Connected Angular clients receive the real-time message via WsService.messages\$.
5. Dashboard store updates (either server-sent payload or client requests latest invoices via API).

Why broker? For horizontal scaling (multiple API instances), broker ensures all nodes see events and can push to connected clients.

Sequence diagram (step-by-step)

1. User clicks Save Invoice -> Angular sends HTTP POST /api/invoices.
 2. .NET API validates & saves -> returns 201.
 3. API publishes invoice.created to RabbitMQ.
 4. Notification worker listens -> sends message to SignalR/WS hub(s).
 5. WS hub broadcasts to all subscribed clients.
 6. Angular WsService receives message -> Store updates -> UI re-renders.
-

Notes to mention in interviews

- Use **SignalR** instead of raw WebSockets for simpler auth, groups, reconnection and scaling features.
 - For heavy realtime load, use a backplane (Redis) or managed services (Azure SignalR Service).
 - Keep business logic in backend — WebSocket pushes events only (not full validation).
 - Consider offline behavior: queue actions locally and sync when online.
 - Security: validate tokens on socket handshake; implement permission checks for broadcast groups.
-

Quick summary — what to memorize for interviews

- Use WebSocketSubject or a singleton service for client-side real-time; handle reconnection and heartbeats.
 - For app state without NgRx, use a BehaviorSubject-backed store with selectors and immutable updates.
-

RxJS Deep Dive

- switchMap cancels old requests (typeahead); mergeMap parallel; concatMap sequential; exhaustMap ignore while busy.
- For scaling realtime in production prefer SignalR + backplane (Redis) or managed SignalR services.
- Show one small code sample (store + ws service) in interview — that demonstrates architecture + practical knowledge.

3 mock interview questions with strong, model answers based on the topics you requested:

- Real-time WebSocket with RxJS
- State management using RxJS (without NgRx)
- Real interview coding tasks
- Architecture understanding

MOCK INTERVIEW QUESTIONS + MODEL ANSWERS

Q1. How will you implement a real-time WebSocket service in Angular using RxJS?

★ Model Answer (Strong & Concise)

In Angular, we implement WebSocket communication using **RxJS WebSocketSubject**, which provides an Observable stream for incoming messages and a Subject for sending data.

Steps I follow:

1. Create a WebSocket service

```
import { Injectable } from '@angular/core';

import { WebSocket, WebSocketSubject } from 'rxjs/webSocket';

@Injectable({ providedIn: 'root' })
export class LiveUpdatesService {
  private socket$: WebSocketSubject<any>;

  constructor() {
    this.socket$ = WebSocketSubject('ws://localhost:8080/live');
  }

  // Receive streamed data
  messages$ = this.socket$.asObservable();

  // Send data
  send(data: any) {
```

RxJS Deep Dive

```
this.socket$.next(data);  
}  
close() {  
  this.socket$.complete();  
}  
}
```

2. Subscribe in a component

```
this.liveService.messages$.subscribe(msg => {  
  console.log('Update:', msg);  
});
```

3. Real-world use cases

- Live billing dashboard
- Inventory count updates
- Invoice payment status updates
- Real-time chat support

★ Why this is a great answer:

- Demonstrates code + concepts
- Shows real project usage
- Shows understanding of RxJS stream-based architecture

Q2. How do you manage state in Angular using RxJS, without NgRx or Signals?

★ Model Answer (Clear & Senior-Level)

State management without NgRx can be done using **BehaviorSubject + Immutable State Pattern**.

What I do:

I create a dedicated service that stores state in a BehaviorSubject and exposes it as an Observable.

Example: Invoice Store Service

```
@Injectable({ providedIn: 'root' })  
export class InvoiceStore {  
  private _invoices = new BehaviorSubject<Invoice[]>([]);  
  invoices$ = this._invoices.asObservable();  
}
```

RxJS Deep Dive

```
addInvoice(newInv: Invoice) {  
  const prev = this._invoices.value;  
  this._invoices.next([...prev, newInv]); // immutable update  
}  
  
updateInvoice(id: number, data: Partial<Invoice>) {  
  const updated = this._invoices.value.map(inv =>  
    inv.id === id ? { ...inv, ...data } : inv  
  );  
  this._invoices.next(updated);  
}  
  
clear() {  
  this._invoices.next([]);  
}  
}
```

How components use it

```
this.store.invoices$.subscribe(list => {  
  this.invoices = list;  
});
```

★ Why this answer impresses interviewers

- Shows understanding of reactive state
- Demonstrates immutability
- Shows how to replace NgRx in small/medium apps
- Clean & reusable architecture

Q3. A client wants “Live Billing Dashboard”. How will you design the architecture using RxJS + WebSocket + .NET 8 backend?

★ Model Answer (Architecture-level)

RxJS Deep Dive

High-level Architecture

[.NET 8 Backend]

|

| (SignalR/WebSocket)

v

[Angular WebSocket Service] --> RxJS stream

|

| BehaviorSubject store

v

[Dashboard Components]

Step-by-step explanation

1. Backend (.NET 8)

- Exposes a **WebSocket / SignalR hub**
- Streams events like:
 - new invoice added
 - payment received
 - inventory updates

2. Angular WebSocket service:

- Uses **RxJS WebSocketSubject**
- Converts server push data → Observable stream

3. State management (RxJS Store)

```
private summary$ = new BehaviorSubject<DashboardSummary>(initialState);
```

```
update(data: DashboardSummary) {  
  this.summary$.next(data);  
}
```

4. UI subscribes to state

```
this.dashboardService.summary$.subscribe(s => {  
  this.summary = s;  
});
```

RxJS Deep Dive

5. Why RxJS?

- Push-based data
- Auto-updating dashboard
- Zero manual refresh
- High performance

★ Final Interview-winning Point

"This approach gives us **real-time, reactive UI** without NgRx complexity. RxJS acts as both state manager and data stream handler."